

 This is a copy of a chat between Claude and Balakrishnan. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

[Report](#)

I like to create a RestAPI using Springboot give me a step by step configuration and coding

[Check for relevant skills](#) >

Here's a complete step-by-step guide to building a REST API with Spring Boot.

Spring Boot REST API — Complete Guide

Step 1: Project Setup (Spring Initializr)

Go to start.spring.io and configure:

Field	Value
Project	Maven
Language	Java
Spring Boot	3.x.x
Packaging	Jar
Java	17 or 21

Dependencies to add:

- Spring Web
- Spring Data JPA
- H2 Database (or MySQL Driver)
- Lombok
- Spring Boot DevTools

Download and unzip, then open in IntelliJ or VS Code.

Step 2: Project Structure

```
src/main/java/com/example/demo/  
├─ controller/  
│   └─ ProductController.java  
├─ service/  
│   └─ ProductService.java  
├─ repository/  
│   └─ ProductRepository.java  
├─ model/  
│   └─ Product.java  
├─ dto/  
│   └─ ProductDTO.java  
└─ DemoApplication.java
```

Step 3: `pom.xml` Dependencies

```
xml  
  
<dependencies>  
  <!-- Web -->  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
  
  <!-- JPA -->  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-jpa</artifactId>  
  </dependency>  
  
  <!-- H2 In-Memory DB (swap for MySQL in prod) -->  
  <dependency>  
    <groupId>com.h2database</groupId>  
    <artifactId>h2</artifactId>  
    <scope>runtime</scope>  
  </dependency>  
  
  <!-- Lombok -->  
  <dependency>  
    <groupId>org.projectlombok</groupId>
```

```
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <!-- DevTools -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Step 4: `application.properties`

```
properties

# Server
server.port=8080

# H2 Database
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# H2 Console (visit: http://localhost:8080/h2-console)
spring.h2.console.enabled=true
```

For **MySQL**, replace the datasource block with:

```
properties

spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
```

```
spring.datasource.password=yourpassword  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Step 5: Model (Entity)

```
java  
  
// model/Product.java  
package com.example.demo.model;  
  
import jakarta.persistence.*;  
import lombok.*;  
  
@Entity  
@Table(name = "products")  
@Data  
@NoArgsConstructor  
@AllArgsConstructor  
@Builder  
public class Product {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false)  
    private String name;  
  
    private String description;  
  
    private Double price;  
}
```

Step 6: DTO (Data Transfer Object)

```
java  
  
// dto/ProductDTO.java  
package com.example.demo.dto;  
  
import lombok.*;
```

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductDTO {
    private String name;
    private String description;
    private Double price;
}
```

Step 7: Repository

```
java

// repository/ProductRepository.java
package com.example.demo.repository;

import com.example.demo.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // JpaRepository gives you: findAll, findById, save, deleteById, etc.
}
```

Step 8: Service Layer

```
java

// service/ProductService.java
package com.example.demo.service;

import com.example.demo.dto.ProductDTO;
import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
```

```
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Product not found with id: " + id));
    }

    public Product createProduct(ProductDTO dto) {
        Product product = Product.builder()
            .name(dto.getName())
            .description(dto.getDescription())
            .price(dto.getPrice())
            .build();
        return productRepository.save(product);
    }

    public Product updateProduct(Long id, ProductDTO dto) {
        Product product = getProductById(id);
        product.setName(dto.getName());
        product.setDescription(dto.getDescription());
        product.setPrice(dto.getPrice());
        return productRepository.save(product);
    }

    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

Step 9: Controller

```
java
// controller/ProductController.java
package com.example.demo.controller;
```

```
import com.example.demo.dto.ProductDTO;
import com.example.demo.model.Product;
import com.example.demo.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    // GET all
    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    // GET by ID
    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    // POST create
    @PostMapping
    public ResponseEntity<Product> create(@RequestBody ProductDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(productService.createProduct(dto));
    }

    // PUT update
    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id, @RequestBody ProductDTO dto) {
        return ResponseEntity.ok(productService.updateProduct(id, dto));
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
    }
}
```

```
        return ResponseEntity.noContent().build();
    }
}
```

Step 10: Main Application Class

java

```
// DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Step 11: Run & Test

Run the app:

bash

```
./mvnw spring-boot:run
```

Test with curl or Postman:

bash

Create a product

```
curl -X POST http://localhost:8080/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","description":"Gaming laptop","price":999.99}'
```

Get all products

```
curl http://localhost:8080/api/products
```

Get by ID


```
curl http://localhost:8080/api/products/1

# Update
curl -X PUT http://localhost:8080/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop Pro","description":"Updated","price":1299.99}'

# Delete
curl -X DELETE http://localhost:8080/api/products/1
```

API Endpoints Summary

Method	Endpoint	Description
GET	<code>/api/products</code>	Get all products
GET	<code>/api/products/{id}</code>	Get product by ID
POST	<code>/api/products</code>	Create new product
PUT	<code>/api/products/{id}</code>	Update product
DELETE	<code>/api/products/{id}</code>	Delete product

Optional Next Steps

- **Validation** — add `spring-boot-starter-validation` and use `@Valid`, `@NotBlank`, `@Min` on DTOs
- **Exception Handling** — add a `@RestControllerAdvice` global exception handler
- **Swagger/OpenAPI** — add `springdoc-openapi-starter-webmvc-ui` for auto-generated API docs at `/swagger-ui.html`
- **Security** — add `spring-boot-starter-security` + JWT for authentication
- **Pagination** — use `Pageable` in your repository for large datasets

Want me to expand on any of these steps or add features like validation, error handling, or JWT auth?

 Turn on web search in Search and tools menu. Otherwise, links provided may not be accurate or up to date.

